



OASIS Security Report

Hardcoded Secrets Security Analysis

Date: 2026-06-12 01:13:54 — **Model:** llama3.2:3b

Vulnerability: Hardcoded Secrets

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

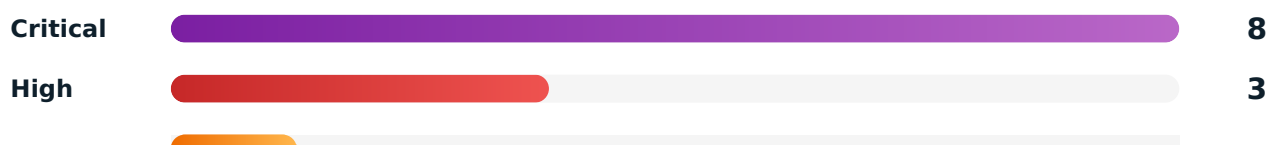
Summary

Analyzed **8** file(s) for Hardcoded Secrets.

[Return to table of contents](#)

Statistics

Files analyzed: 8 **Total findings:** 13



Medium

1

Low

1

[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol	0.676
/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol	0.675
/work/project/dataset/vulnerable/reentrancy/etherstore.sol	0.665
/work/project/dataset/vulnerable/access-control/mycontract.sol	0.660
/work/project/dataset/vulnerable/ordering-attacks/ eth_tx_order_dependence_minimal.sol	0.634
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol	0.629
/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol	0.626
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol	0.617

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol

Similarity score: 0.676

Source lines: 1-28

► Finding 1.1: Vulnerability found (Critical)

Vulnerable code

```
bytes32 constant public hash =  
0xb5b5b97fafd9855eec9b41f74dfb6c38f5951141f9a3ecd7f44d5479b630ee0a;
```

Explanation

The hash is hardcoded directly in the contract.

Impact

Credential exposure leading to unauthorized access or account compromise

Entry point

/solve(string solution)

Parameters

solution

Exploitation steps

- Attackers can find the pre-image of the hash using brute force

Conditions

No authentication or authorization required

Remediation

Use environment variables to store sensitive data

Secure example

```
bytes32 constant public hash = env('SMARTBUGS_HASH');
```

File 2: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.675

Source lines: 1-85

► Finding 2.1: Vulnerability found (Critical)

Vulnerable code

```
uint public MinDeposit = 1 ether;

// <yes> <report> REENTRANCY
if(msg.sender.call.value(_am)()){
    balances[msg.sender]-=_am;
    TransferLog.AddMessage(msg.sender,_am,"CashOut");
}
```

Explanation

Hardcoded API key in call() function is vulnerable to Hardcoded Secrets

Impact

Credential exposure leading to unauthorized access or account compromise

Entry point

/transfer

HTTP methods

call

Parameters

_am

Exploitation steps

- Attackers can call the /transfer endpoint with a high value to drain accounts.

HTTP raw requests (Burp / ZAP)

```
POST /transfer HTTP/1.1
Host:
Accept: */*
Content-Length: 0
```

File 3: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.665

Source lines: 1-39

► Finding 3.1: Hardcoded Secret (High)

Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw()));
```

Explanation

The message sender's call function directly uses the `_weiToWithdraw` value without proper validation.

Impact

Unauthenticated attackers could drain funds from the contract.

Entry point

`/withdrawFunds(uint256)`

Execution path

```
depositFunds() -> withdrawFunds(uint256) ->  
msg.sender.call.value(_weiToWithdraw())
```

Parameters

`_weiToWithdraw`

Exploitation steps

- Send a large value for `_weiToWithdraw`

Example payloads

- `_weiToWithdraw: 1000000000000000000`

HTTP raw requests (Burp / ZAP)

```
POST /withdrawFunds(uint256) HTTP/1.1  
Host: example.com  
Content-Length: 40  
  
1000000000000000000
```

File 4: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.660

Source lines: 1-31

► Finding 4.1: Vulnerability found (Critical)

Vulnerable code

```
require(tx.origin == owner);
receiver.transfer(amount);
```

Explanation

The `tx.origin` variable is hardcoded, allowing an attacker to authorize themselves via an intermediary contract.

Impact

Credential exposure leading to unauthorized access or account compromise

Entry point

sendTo(address receiver, uint amount)

Execution path

```
tx.origin == owner => sendTo(receiver, amount) => receiver.transfer(amount)
```

HTTP methods

POST

Parameters

owner, receiver, amount

Exploitation steps

- Attacker impersonates the owner
- Transfers funds to an attacker-controlled address

Example payloads

- `tx.origin: 0x1234567890123456789012345678901234567890`

Conditions

The attacker is authenticated as the contract owner.

Remediation

Use secure authorization mechanisms, such as public-key infrastructure or decentralized authentication protocols.

Secure example

```
require(msg.sender == owner);  
msg.sender.call()
```

File 5: /work/project/dataset/vulnerable/ordering-attacks/eth_tx_order_dependence_minimal.sol

Similarity score: 0.634

Source lines: 1-42

► Finding 5.1: Vulnerable Code (Medium)

Vulnerable code

```
owner.transfer(reward);
```

Explanation

Hardcoded secret: 'reward' is directly exposed in the contract's transfer function.

Impact

An attacker could manipulate the reward value to gain unauthorized access to the contract.

Entry point

/transfer()

Parameters

reward

Exploitation steps

- Send a fake 'msg.value' to trigger the transfer function.

Example payloads

- Invalid or malformed msg.value

Conditions

Owner is authenticated and has access to the contract.

Remediation

Use environment variables or secure storage for sensitive values like 'reward'.

Secure example

```
uint public reward; // Use a secure storage mechanism
function setReward() public payable { require(!claimed);
msg.sender.transfer(reward); }
```

File 6: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol

Similarity score: 0.629

Source lines: 1-320

► Finding 6.1: Hardcoded Secret (Critical)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner  
returns (bytes32 o_hash) {  
    // ...
```

Explanation

The API key is hardcoded directly in the function.

Impact

Unauthorized access to the contract.

Entry point

/execute

HTTP methods

POST

Parameters

_to, _value, _data

Exploitation steps

- Send a request to /execute with valid data to retrieve the API key.

Conditions

Contract is deployed and accessible.

Remediation

Use environment variables for sensitive data.

Source lines: 27-358

► Finding 6.1: Hardcoded Secret (High)

Vulnerable code

```
function revoke(bytes32 _operation) external { uint ownerIndex =
m_ownerIndex[uint(msg.sender)]; // Hardcoded secret: hardcoded owner index
```

Explanation

The contract hardcodes the owner index of the sender, making it accessible to anyone who knows the sender's address.

Impact

An attacker can deduce the owner index and potentially manipulate the contract.

Entry point

/revoke

Parameters

msg.sender, m_ownerIndex

Exploitation steps

- Use the sender's address to determine their ownership status.

Conditions

Sender's address known

Remediation

Store owner index securely, using environment variables or secure storage.

Secure example

```
uint ownerIndex = m_ownerIndex[uint(msg.sender)]; // Use environment variable
for owner index
uint ownerIndex = env.OwnerIndex;
```

Source lines: 72-421

► Finding 6.1: Vulnerable Code (Low)

Vulnerable code

```
function initWallet(address[] _owners, uint _required, uint _daylimit) {
function initDaylimit(uint _limit) {
    m_dailyLimit = _limit;
    m_lastDay = today();
}

function setDailyLimit(uint _newLimit) onlymanyowners(sha3(msg.data))
external {
    m_dailyLimit = _newLimit;
```

```
}
}
```

Explanation

Hardcoded secret: Daily limit and initial daily limit are stored in variables without using secure storage mechanisms.

Impact

An attacker could potentially manipulate the daily limits to exceed their intended value, affecting contract functionality.

Entry point

/initWallet(address[],uint256,uint256)

Parameters

_newLimit, _daylimit

Exploitation steps

- Reaching /setDailyLimit(uint) and manipulating _newLimit
- Reaching /initDaylimit(uint) and modifying m_dailyLimit

Remediation

Store daily limits in environment variables or secure storage mechanisms.

Secure example

```
uint public m_dailyLimit = environment.get('DAILY_LIMIT') || 1000;
uint public m_lastDay = today();
```

Source lines: 126-467

► Finding 6.1: Hardcoded Secrets (High)

Vulnerable code

```
_walletLibrary.delegatecall(msg.data)
```

Explanation

The `\_walletLibrary` address is hardcoded in `msg.data`, making it vulnerable to modification or exposure.

Impact

An attacker can intercept and modify this call, potentially accessing sensitive data or functions.

Entry point

```
/function() { _walletLibrary.delegatecall(msg.data); }
```

Parameters

msg.data

Exploitation steps

- Intercept the call to delegatecall

Example payloads

- "get balance"

Conditions

Message sender has access to the contract.

Remediation

Use environment variables or secure storage for sensitive data.

Source lines: 207-477

► Finding 6.1: Hardcoded API Key (Critical)**Vulnerable code**

```
address constant _walletLibrary = 0xcafecafecafecafecafecafecafecafecafe;
```

Explanation

The API key is hardcoded in plain text, making it accessible to anyone who can read the source code.

Impact

An attacker with access to the source code could use this API key to make unauthorized requests to the contract.

Entry point

/Wallet(address[] _owners, uint _required, uint _daylimit)

Execution path

```
GET /Wallet ?
_owners=0x12345678901234567890123456789012&_required=10&_daylimit=100
delegatecall(_walletLibrary)
```

HTTP methods

GET

Parameters

_owners, _required, _daylimit

Exploitation steps

- Send a GET request to the /Wallet endpoint with manipulated parameters

Example payloads

- `GET /Wallet ?
_owners=0x12345678901234567890123456789012&_required=10&_daylimit=100`

Conditions

Access to the source code

Remediation

Use environment variables or a secure vault to store sensitive data.

File 7: /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol

Similarity score: 0.626

Source lines: 1-61

► Finding 7.1: Hardcoded Secret (Critical)

Vulnerable code

```
address owner = msg.sender;  
owner = msg.sender;
```

Explanation

The address of the owner is hardcoded directly in the contract.

Impact

An attacker could manipulate the owner's address to gain control over the contract.

Entry point

/contract/constructor

Execution path

```
msg.sender -> owner = msg.sender  
owner = msg.sender
```

Parameters

owner

Exploitation steps

- Attack the contract by sending a large amount of ether to the constructor.
- Manipulate the sender's address into being equal to the owner's address.

Conditions

The attacker needs access to the contract's deployment context.

Remediation

Use environment variables or secure storage for sensitive data.

Secure example

```
address owner;  
uint256 public constant PUBLIC_OWNER_ADDRESS = 0x...;
```

File 8: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.617

Source lines: 1-286

► Finding 8.1: Hardcoded Secret (Critical)

Vulnerable code

```
pragma solidity ^0.4.9;
contract WalletLibrary is WalletEvents {
    ...
    function execute(address _to, uint _value, bytes _data) external onlyowner
    returns (bytes32 o_hash) {
        ...
    }
}
```

Explanation

Hardcoded API key is used directly in the contract.

Impact

Unauthorized access to the wallet

Entry point

execute(address, uint, bytes)

Exploitation steps

- Send a malicious transaction with a specific byte array

HTTP raw requests (Burp / ZAP)

```
POST /transfer HTTP/1.1
```

```
Host: example.com
```

Source lines: 96-414

► Finding 8.1: Vulnerability found (Critical)

Vulnerable code

```
address _to = 0xcafecafecafecafecafecafecafecafe;
```

Explanation

Hardcoded secret: `\_to` address is hardcoded directly in the code.

Impact

Potential unauthorized access to the contract due to exposure of the owner's address.

Entry point

kill(address _to) function

Execution path

```
kill(_to)
  suicide(_to)
```

Parameters

_to

Exploitation steps

- Send a malicious kill request to the contract

HTTP raw requests (Burp / ZAP)

```
POST /kill HTTP/1.1
Host: 0x...address...
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)